

Function Transformer - Complete Notes

What is a Function Transformer?

`FunctionTransformer` is a preprocessing tool in Scikit-learn used to apply mathematical functions to numerical features.

Its main purpose is to:

- Reduce skewness
- Make data closer to a normal distribution
- Reduce the effect of outliers
- Improve model performance

It is especially useful for algorithms that assume normally distributed data, such as:

- Linear Regression
- Logistic Regression
- Naive Bayes
- KNN
- SVM
- PCA

Tree-based models usually do not require these transformations:

- Decision Tree
- Random Forest
- XGBoost

Import

```
from sklearn.preprocessing import FunctionTransformer
```

Basic Syntax

```
transformer = FunctionTransformer(function=np.log1p)

X_transformed = transformer.fit_transform(X)
```

What is Skewness?

Skewness describes the asymmetry of data distribution.

Normal Distribution

- Mean $\approx$ Median $\approx$ Mode
- Bell-shaped curve
- Skewness $\approx$ 0

Right-Skewed (Positive Skew)

- Long tail on the right side
- Few very large values

Examples:

- Income
- House prices
- Population

Left-Skewed (Negative Skew)

- Long tail on the left side
- Few very small values

Examples:

- Exam scores with easy questions
 - Customer satisfaction ratings
-

How to Check Whether Data is Normal?

1. Histogram

```
df["column"].hist()
```

2. Box Plot

```
sns.boxplot(x=df["column"])
```

3. Skewness Value

```
df["column"].skew()
```

Interpretation:

- -0.5 to 0.5 $\rightarrow$ Approximately normal
- 0.5 $\rightarrow$ Right skewed

- $< -0.5 \rightarrow$ Left skewed

4. Q-Q Plot

```
import scipy.stats as stats
import matplotlib.pyplot as plt

stats.probplot(df["column"], dist="norm", plot=plt)
plt.show()
```

If points follow a straight line, data is approximately normal.

Types of Transformations

1. Log Transformation

Formula:

$$x' = \log(x)$$

Usually:

$$x' = \log(x + 1)$$

Python:

```
np.log1p(x)
```

Use When:

- Data is right skewed
- Values are positive

Examples:

- Income
- Sales
- Population

Advantages:

- Compresses large values
- Reduces skewness

Limitations:

- Cannot handle negative values
 - `log(0)` is undefined
-

2. Reciprocal Transformation

Formula:

$$x' = 1 / x$$

Python:

$$1 / x$$

Use When:

- Data is highly right skewed

Advantages:

- Strongly reduces skewness

Limitations:

- Cannot handle zero values
 - Sensitive to small values
-

3. Square Root Transformation

Formula:

$$x' = \sqrt{x}$$

Python:

$$\text{np.sqrt}(x)$$

Use When:

- Data is moderately right skewed
- Count data

Examples:

- Number of customers
- Number of accidents

Advantages:

- Handles zero values
- Less aggressive than log transform

Limitations:

- Cannot handle negative values

4. Square Transformation

Formula:

$$x' = x^2$$

Python:

```
np.square(x)
```

Use When:

- Data is left skewed

Advantages:

- Stretches larger values
- Reduces left skewness

Limitations:

- Increases impact of outliers

Which Transformation Should I Use?

Data Type	Skewness	Transformation
Mild right skew	0.5 to 1	Square Root
Moderate right skew	1 to 2	Log
Extreme right skew	> 2	Reciprocal

Data Type	Skewness	Transformation
Left skew	< -0.5	Square

Example Using FunctionTransformer

Log Transform

```
import numpy as np
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log1p)

X_new = log_transformer.fit_transform(X)
```

Square Root Transform

```
sqrtransformer = FunctionTransformer(np.sqrt)

X_new = sqrtransformer.fit_transform(X)
```

Square Transform

```
square_transformer = FunctionTransformer(np.square)

X_new = square_transformer.fit_transform(X)
```

Reciprocal Transform

```
reciprocal_transformer = FunctionTransformer(
    lambda x: 1 / x
)

X_new = reciprocal_transformer.fit_transform(X)
```

Using with ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import FunctionTransformer

trf = ColumnTransformer([
    ("log", FunctionTransformer(np.log1p), ["income"]),
```

```
 ("sqrt", FunctionTransformer(np.sqrt), ["population"])  
 ], remainder="passthrough")
```

Important Notes

- Apply transformations only to numerical columns.
- Fit the transformer only on training data.
- Use the same fitted transformer on test data.
- Check the distribution before and after transformation.
- Tree-based algorithms usually do not need transformations.
- Avoid applying transformations blindly.

Workflow:

```
Visualize Data  
  ↓  
Check Skewness  
  ↓  
Choose Transformation  
  ↓  
Apply FunctionTransformer  
  ↓  
Train Model  
  ↓  
Evaluate Performance
```